Developer Specification

Production-Grade Structured Text Logic Generation Stack

Goal

Generate complete, deployable, production-grade Structured Text logic every time by forcing the system through a deterministic engineering pipeline:

Documents
→ extraction
→ normalization
→ plant graph
→ control loop discovery
→ engineering validation
→ logic completion
→ ST generation
→ ST parsing / verification
→ IO mapping
→ simulation validation
→ runtime packaging

This is the correct architecture because it prevents the code generator from guessing too early.

---

1. Open-Source Tools Required

Core orchestration + API

FastAPI

Use as the backend control plane.

Used for:

- upload endpoints

- parse endpoints
- generate-logic endpoints
- validate endpoints
- export endpoints

Why:

- lightweight
- easy Python service wiring
- already fits your architecture

Document ingestion

PyMuPDF

Use for reading uploaded PDFs and extracting embedded text.

Used for:

- P&ID text extraction
- equipment list parsing
- control narrative ingestion
- raw document text extraction

Connection:

- React uploads file
- FastAPI stores file
- Python service reads with PyMuPDF

OCR for drawings / scanned docs

Tesseract OCR

Use when P&IDs or drawings are image-based.

Used for:

- tag extraction
- instrument labels
- pump / valve identifiers
- text from scanned sheets

Connection:

- file saved by FastAPI
- OCR service runs on image pages
- extracted text pushed into metadata pipeline

Metadata cleaning + normalization

Pandas

Use for engineering data normalization.

Used for:

- tag cleanup
- duplicate removal
- field standardization
- missing-field normalization
- signal table construction

Examples:

- LT101 → LT-101
- P101 → P-101
- remove duplicates
- normalize instrument prefixes

Messy tag reconciliation

RapidFuzz

Use for fuzzy matching when documents are incomplete, inconsistent, or dirty.

Used for:

- LT101 vs LT-101 vs LIT101 matching

- PUMP101 vs P-101 matching
- resolving near-duplicate equipment tags
- legacy naming cleanup

Why:

- real industrial docs are messy
- this prevents fragmented plant graphs

This is explicitly aligned with the robustness / reconciliation logic already described in your uploaded architecture.

Plant system model

Neo4j

Use as the canonical plant knowledge graph.

Used for:

- equipment nodes
- sensor nodes
- actuator nodes
- process unit relationships
- signal paths
- control dependencies

Why:

- all downstream logic should come from graph relationships, not direct prompt guessing
- your architecture already centers everything around the plant graph

---

Process dynamics layer

OpenModelica

Use to attach behavior to plant topology before logic generation.

Used for:

- tank level dynamics
- pump flow curves
- valve response timing
- simple physics constraints

Why:

- a P&ID is static
- control logic quality goes up when the system knows how the plant behaves, not just what exists in it

---

Control strategy synthesis

Durable Rules

Use as the deterministic control rule engine.

Used for:

- infer control strategies
- infer alarms
- infer interlocks
- infer sequences
- infer startup / shutdown rules
- infer fallback behavior from plant structure

Why:

- do not let the LLM directly write final PLC logic
- first generate explicit engineering rules from plant structure

---

Dataset sanity checks

jsonschema

Use for engineering validation before code generation.

Used for:

- required sensor checks
- actuator linkage checks
- loop completeness checks
- duplicate tag checks

- missing signal checks
- broken equipment path checks

Why:

- catches missing engineering structure before code generation
- already defined as Step 4.5 in your architecture

Structured Text syntax parsing

Tree-Sitter IEC 61131-3 / Structured Text grammar

Use to parse generated ST into an AST.

Used for:

- syntax validation
- token parsing
- block completeness checks
- missing END_IF / END_CASE / END_PROGRAM detection
- routine structure validation
- parse-before-deploy enforcement

Why:

- this is the hard gate that prevents pseudo code and malformed ST from passing through

- your editor / export stack already points to Tree-Sitter for ST parsing and validation

---

Logic templating

Jinja2

Use for deterministic ST generation from validated engineering objects.

Used for:

- pump routine templates
- valve routine templates
- PID templates
- alarm templates
- interlock templates
- startup / shutdown templates
- vendor-neutral ST file templates

Why:

- final code should come from templates + validated data model, not freeform generation
- this also aligns with your export engine design

Database for structured engineering metadata

PostgreSQL

Use for non-graph structured data.

Used for:

- control loop table
- tag tables
- IO mapping table
- project state
- validation issues
- generation audit records

Why:

- graph for relationships, relational DB for engineering tables
- already in your architecture

IO generation

Python + Pandas + PostgreSQL

Use this stack for deterministic IO mapping generation.

Used for:

- signal classification
- IO type assignment

- PLC slot / channel assignment
- tag-to-channel mapping
- mapping export

Why:

- logic is not production-grade if tag / IO structure is incomplete
- your uploaded IO engine already defines this correctly

PLC runtime validation

OpenPLC Runtime

Use to compile / load / execute generated ST in a real runtime target.

Used for:

- project creation
- ST import
- runtime execution
- IO binding test
- deployment readiness check

Why:

- generation is not complete until the logic runs in a target runtime

- already defined in your architecture

Virtual commissioning

SimPy

Use for lightweight process simulation object generation.

OpenModelica

Use for more realistic dynamic validation.

Used together for:

- process response testing
- overflow conditions
- sequence validation
- control loop behavior testing
- alarm trigger verification

Why:

- logic quality must be validated against plant behavior, not just syntax

---

Version / audit safety

Git

GitPython

Dolt

Use for automatic traceability of generated logic and plant model changes.

Used for:

- auto commits on generation
- plant model snapshots
- logic version history
- metadata versioning
- rollback

Why:

- production systems require reproducibility and auditability
- your uploaded versioning architecture already defines this layer clearly

---

2. Correct Step-by-Step Sequence

Step 1 — Ingest documents

Tools:

- FastAPI
- PyMuPDF

- Tesseract OCR

Flow:

React UI
→ FastAPI upload
→ file saved
→ PyMuPDF extracts text
→ Tesseract extracts text from scans/drawings
→ raw metadata output

Output:

- raw engineering text
- OCR text
- initial extracted tags

Step 2 — Normalize engineering metadata

Tools:

- Pandas
- RapidFuzz

Flow:

raw extracted metadata
→ pandas cleanup
→ normalize tags
→ deduplicate devices
→ fuzzy-match messy names
→ clean engineering dataset

Output:

- normalized equipment list
- normalized instrument list
- resolved tag dictionary

Step 3 — Build plant graph

Tools:

- Neo4j
- Python Neo4j driver

Flow:

clean engineering dataset
→ create equipment nodes
→ create sensor nodes
→ create actuator nodes
→ create process edges
→ create signal edges
→ plant graph

Output:

- canonical plant graph in Neo4j

Step 4 — Attach dynamics

Tools:

- OpenModelica

Flow:

normalized metadata
→ generate simple process model templates
→ attach dynamics to process units
→ enrich plant graph with behavior

Output:

- topology + behavior model

Step 5 — Validate engineering completeness

Tools:

- jsonschema
- Python

Flow:

plant dataset
→ schema validation
→ check missing sensors
→ check broken paths
→ check missing actuator signals
→ check duplicate tags
→ validation report

Output:

- validated plant model
- warnings / errors to UI

Step 6 — Discover control loops

Tools:

- Neo4j
- Python
- PostgreSQL

Flow:

plant graph
→ query Sensor → Process → Actuator relationships
→ infer loops
→ create control_loops records
→ store in PostgreSQL

Output:

- control loop table
- inferred strategies
- loop confidence values

Step 7 — Logic Completion Engine

Tools:

- Durable Rules
- Pandas
- PostgreSQL
- Neo4j

This is the step that guarantees completeness.

Flow:

plant graph
+ control loops
+ engineering validation results
→ infer missing alarms
→ infer missing interlocks
→ infer startup / shutdown sequences
→ infer equipment routines
→ infer default command/status/fault tags
→ generate full logic model

This layer must create these objects even when the original documents do not include them:

- equipment routines
- alarm groups
- interlock rules
- startup sequence
- shutdown sequence
- command/status/fault tags
- setpoint and output references
- basic fallback operating logic

Output:

- completed logic model
- no empty routines
- no partial loop definitions

This is the missing “production-grade accuracy” layer.

Step 8 — Generate Structured Text deterministically

Tools:

- Jinja2
- Python

Flow:

completed logic model
→ map each object to ST templates
→ render equipment files
→ render control loop files
→ render interlock files
→ render alarm files
→ render main.st

Output folder should follow your existing structure:

/control_logic
main.st
equipment/
control_loops/
sequences/
interlocks/
alarms/
utilities/

That exact project structure is already defined in your uploaded architecture and is the correct one.

Step 9 — Parse and verify the generated ST

Tools:

- Tree-Sitter Structured Text grammar

Flow:

generated .st files
→ parse to AST
→ validate syntax
→ validate block closure
→ validate program/function block structure
→ reject malformed files

Hard fail conditions:

- parse failure
- missing END_IF
- missing END_PROGRAM
- malformed CASE
- undefined routine block structure
- empty template sections left in code

Output:

- syntactically valid ST only

Step 10 — Build IO mapping

Tools:

- Pandas
- PostgreSQL
- Python
- OpenPLC runtime API

Flow:

completed logic model
+ plant graph
→ classify signal types
→ assign AI / AO / DI / DO
→ assign PLC slot / channel
→ generate mapping table
→ write to PostgreSQL
→ bind to runtime config

Output:

- complete IO mapping
- runtime-ready signal map

Step 11 — Run runtime validation

Tools:

- OpenPLC Runtime

Flow:

validated ST
+ IO mapping
→ create OpenPLC project
→ import ST files
→ apply IO config
→ start runtime
→ check runtime load success

Output:

- compiled / loaded PLC runtime

Step 12 — Run virtual commissioning

Tools:

- SimPy
- OpenModelica
- OpenPLC Runtime

Flow:

runtime logic
+ dynamic process model
→ simulate normal operation
→ simulate faults
→ simulate alarm cases
→ simulate edge conditions
→ record pass/fail

Minimum required test set:

- startup sequence
- shutdown sequence
- high level alarm
- low level alarm
- actuator failure
- stuck valve
- pump failure
- sensor out-of-range

Output:

- simulation-validated control logic

Step 13 — Version the result automatically

Tools:

- Git
- GitPython
- Dolt

Flow:

validated plant graph

+ generated logic
+ IO mapping
+ simulation results
→ auto commit
→ snapshot
→ metadata version record

Output:

- reproducible engineering state
- rollback capability
- audit trail

---

3. How They Connect Together

System wiring

React UI
→ FastAPI

FastAPI
→ PyMuPDF
→ Tesseract
→ Pandas normalization
→ RapidFuzz reconciliation
→ Neo4j graph builder
→ OpenModelica dynamics generator
→ JSON Schema validation
→ Control loop engine
→ Durable Rules logic completion engine
→ Jinja2 ST generator
→ Tree-Sitter ST validator
→ Pandas/PostgreSQL IO mapping engine
→ OpenPLC runtime
→ SimPy/OpenModelica virtual commissioning
→ Git/GitPython/Dolt versioning

---

4. Non-Negotiable Rules for Accuracy

Your developer should follow these rules exactly.

Rule 1

Do not generate final ST directly from the LLM.

Correct:

LLM or parser helps extract structure
→ deterministic rules complete the model
→ templates generate ST

Wrong:

P&ID text
→ LLM writes final PLC logic

Rule 2

The code generator must only operate on a completed engineering object model.

That model must contain:

- devices
- tags
- signals
- control loops
- interlocks
- alarms

- sequences
- IO types

No direct freeform generation from raw text.

Rule 3

Every generated file must pass Tree-Sitter parse validation before being accepted.

No exceptions.

Rule 4

Every generated project must pass at least one runtime load in OpenPLC.

No “looks correct” approval.

Rule 5

Every generated project must pass simulation checks against the plant model.

Syntax-valid code is not enough.

---

5. Minimal Folder / Service Structure

```
/backend
  /api
  /services
    document_ingest.py
    ocr_extract.py
    normalize_tags.py
    reconcile_tags.py
```

```
    plant_graph_builder.py
    process_dynamics.py
    engineering_validator.py
    control_loop_engine.py
    logic_completion_engine.py
    st_generator.py
    st_validator.py
    io_mapping_engine.py
    runtime_deployer.py
    virtual_commissioning.py
    version_manager.py
  /templates
    /equipment
    /control_loops
    /interlocks
    /alarms
    /sequences
  /schemas
    plant_model.schema.json
    control_loop.schema.json
    logic_model.schema.json
```

This fits your uploaded modular architecture and keeps the implementation clean.

---

6. Final Correct Tool List

Use exactly this stack for the logic generation reliability layer:

- FastAPI
- PyMuPDF
- Tesseract OCR
- Pandas
- RapidFuzz
- Neo4j

- OpenModelica
- Durable Rules
- jsonschema
- Tree-Sitter IEC 61131-3 / ST grammar
- Jinja2
- PostgreSQL
- OpenPLC Runtime
- SimPy
- Git
- GitPython
- Dolt

---

7. Final One-Line Summary for Your Developer

Do not let the system generate Structured Text from documents directly; first force documents into a validated plant graph and completed engineering logic model, then generate ST from deterministic templates, then hard-validate it through Tree-Sitter, OpenPLC runtime loading, and simulation.

That is the correct production-grade architecture.

Your uploaded docs already provide most of the surrounding architecture for this approach, including the plant graph, validation, ST editor/parser, IO mapping, control loop engine, and versioning layers.